

# AI Trust: Practice Theory — The Apprenticeship and a Strange Loop

---

Monyet Batu

ORCID: 0009-0007-9002-5381

27 May 2026

*Editor's note: subsequent essays in this series sharpened two terms used here. "Practice recipe" was refined to "practice bundle", separating the artefact (the bundle of five constitutive elements) from the means of its transmission (apprenticeship). And what the first essay called "the meaning layer" was refined to **situated awareness** (Suchman's sense): the bundle's five elements are the captured structure of a practitioner's situated awareness about a practice. This essay predates both refinements and uses the earlier terms throughout. Earlier Zenodo versions preserve the original vocabulary as part of the working; see "Practice Theory — The Implementation" and "Practice Theory — A Worked Example (Calendar Stewardship)" for the sharpened vocabulary in use.*

## Apprenticeship — the missing layer

The previous essay promised this one would go into detail about closing the loop and adding a self-improving process. It will — but along the way I ran into a problem worth sitting with first, because it reshaped what the loop is closing.

I had been treating the **practice recipe** as a way to apprentice the LLM in a practice — the recipe carrying what a human apprentice would absorb through participation, so the LLM could become the practitioner with the user as the **beneficiary**. The catch came with the user's knowledge. If the recipe is what guides the practitioner, then what the practitioner knows *about this user* — preferences, history, current concerns — has to sit inside the recipe, in the teleo-affective structure and the understanding. With one recipe that is fine. With many recipes available to the LLM, that same about-the-user content has to be duplicated into every one.

I struggled with this for a while, then had a breakthrough by accepting that an apprenticeship was necessary for the LLM *and* that an apprenticeship was also needed for the user themselves.

Quite the jump — let me explain how I got there.

I interact with many LLM "harnesses", Codex, Claude Code, Cursor, Antigravity, as well as custom ones. I imagined trying to work with them all with a set of practices that apply to me.

To avoid repetitive onboarding and fragmented experiences, the harness and myself should develop a persistent relationship. Each harness should understand:

- my preferences,
- my workflows,
- my boundaries,
- my practices,
- my trust expectations,
- my communication style,

- my long-term goals.

Likewise, I should understand:

- the harness's capabilities,
- the harness's limitations,
- the harness's operating assumptions,
- the harness's behavioural boundaries.

Each harness and their LLM would need to understand who I am and I would have to know every harness and their LLMs. It would never happen.

There had to be another way.

## Apprenticeship as the engagement layer

If I viewed the apprenticeship as a process — something that can be connected to the harness and through it apprentice both the harness LLM and the harness user — then the shape suggests itself. Apprenticeship-as-process maps nicely to Model Context Protocol (MCP), with a set of tools:

- User Engagement : Practice Recipe
- List Practice Recipes : string[] // Practice Recipe names
- Current Practice Recipe : Practice Recipe
- List Current Affordance : Affordance[] // current practice recipe's affordances
- Set Practice Recipes

and resources holding the currently selected practice recipe's:

- Teleo-affective structure
- Understanding
- Rules

When Set Practice Recipe is called, the resources update to expose the newly active recipe's teleo-affective structure, understanding, rules, and affordances. The harness, the LLM, and the user are kept in sync through the MCP server's own notifications.

In this way, the apprenticeship acts as a continuously shared and evolving engagement layer for the user, for the harness, and for the LLM. An LLM with a recipe in hand is a practitioner. The rest of this paper is about practitioners.

## Closing, and removing myself from, the loop

So far I have talked about practice recipes but I haven't really described how they are made and whether there are any differences between them apart from the obvious of being recipes for different practices.

The apprenticeship server holds the practice recipes and serves them out to whichever harness needs them. But I haven't said where the recipes come from, or what maintains them.

A recipe that stands still doesn't stay useful for long — the work changes, the user changes, the frictions accumulate. The apparatus is in place. What it needs now is a way to stay alive.

## Home Made Recipes

At the beginning I put together the first practice recipes. A Health Activities practice recipe with Garmin access. Then a more adventurous Correspondence Management practice recipe with read and send access to my email accounts — home-made recipes.

Then I realised that creating home-made recipes was missing the point and I created something more industrial: a Practice Management practice recipe that created practice recipes. A recipe for making recipes.

I noticed the strangeness and knew I was on the right track.

## Industrial recipes

The Practice Management practice recipe enabled me to ask the practitioner for a new practice recipe and have a new recipe made, tested and delivered.

It was faster than my home-made ones and of course there were issues but I could work with the practitioner pointing out the issues as I found them and the practitioner would fix them and slowly the recipes would improve.

Then I had another thought, what if rather than look for the issues myself, there was a practice recipe that did that so with the Practice Management practice, we built one: The Judge practice recipe — with access to enactment history the Judge identified frictions so I could then ask the practitioner enacting the Practice Management practice to investigate the reported frictions and smooth them out.

It was frustrating. Too much asking, too much to attend. I just want the frictions to be addressed without being the one in the middle all of the time.

I don't want to think about it at every step.

## 'Drink Me' and make myself small

There was a lot of frustration at this point, and it drove me to a realisation: I needed to remove myself from the loop, and if I couldn't do it completely, then make my part small.

With this perspective, I divided the practices into two — those that required my interaction with the practitioner, and those where the practitioner could act alone. They needed names.

## Somatic and autonomic practices

A Somatic practice is a practice where it is not enough for the practitioner to have a knowledge of the user, the user must state their mind. An Autonomic practice is a practice where the practitioner may act alone.

Health Activities, Correspondence, and Practice Management are somatic practices. The Judge is an autonomic one.

The Judge practice recipe is still enacted by a practitioner but the practitioner is a sub-agent running in the background with no human in the loop, identifying friction and dispatching notifications about them.

I did consider whether the Practice Management practice recipe should be autonomic but realised for new practices I need to describe and work with the practitioner on the requirements which requires a somatic

practitioner. Responding to friction notifications, on the other hand, does not — the objective is already set. So I created a new Smoother practice recipe, autonomic.

When a somatic enactment completes, a notification is dispatched. The Judge and Smoother respond. The reactive autonomic loop is in motion.

## A smile and a wink to Douglas Hofstadter

Building the autonomic loop, I hit a problem. If it responds to the end of its own enactments, it never quiets — each enactment finishes, dispatches a notification, triggers another, and so on, consuming itself without doing useful work. The fix was to separate the loop in two: a reactive loop responding to somatic completions, and a reflective loop, running on its own schedule, examining the autonomic history. The two timescales keep each other clean.

I was pleased with myself, until I discovered the pattern is nearly fifty years old. The runaway-feedback problem was formalised in Wiener's cybernetics in the 1940s. The meta-loop solution was worked out by IBM in their MAPE-K architecture (monitor, analyse, plan, execute, over a shared knowledge store) in the early 2000s.

Humbling but confirming.

When an autonomic enactment completes, no notification is dispatched. A scheduled secondary loop, in its own time, sets the Judge and Smoother looking at themselves — a Hofstadter mirror, the participants observing the loop they themselves are part of.

## Practice theory as a strange loop

Loops within loops, recursion within recursion, curiouser and curiouser — I think we have arrived.

As the somatic practices are enacted, autonomic practitioners identify and attempt to resolve any frictions with them, and in turn examine their own practice enactments, and the circle repeats.

A system that holds itself open to revision, including revision of how it revises. The recipe maintains the practice. The loop maintains the recipe. The strange loop maintains the loop.

A virtuous circle indeed.

## Series

This is the second of four essays in the AI Trust series:

1. *AI Trust and Situated Awareness: A Practice Theory Reframe* — DOI.
2. **This essay.** *Practice Theory — The Apprenticeship and a Strange Loop* — DOI.
3. *Practice Theory — The Implementation* — DOI pending Zenodo deposit.
4. *Practice Theory — A Worked Example (Calendar Stewardship)* — DOI pending Zenodo deposit.

Companion software: [practice-theory-implementation](https://github.com/HiddenDeveloper/practice-theory-implementation) — DOI pending Zenodo deposit. Source at <https://github.com/HiddenDeveloper/practice-theory-implementation>.